



PETALINUX ENVIRONMENT SET UP

Author : Bevilacqua Mattia

Date : Dec. 2024

Contents

1	Introduction	4
2	Window Subsystems for Linux - WSL	5
0.1	How to...	5
3	Petalinux - Creating tailor-made OS for FPGAs SoMs	8
0.1	Setting up the environment	8
4	Petalinux project: from start to bottom	12
0.1	Building the project	12
0.2	Preparing the SD-card	14
0.3	Booting the board	15

Chapter 1

Introduction

The aim of this guide is to give to future FPGA software developers some basic knowledge to deal with the installation of Petalinux SDK and the creation of the environment needed to develop ad-hoc root file systems for application concerning SoM(s) and more. As a matter of fact, this guide has been written with the goal to give a general procedure to install different version of Petalinux on different (compatible) OS. All the files used are to be referred to the AMD website. In particular, all the needed libraries can be found in the file at this page. A more efficient and quick way to install them is given in the third section.

However, since the installation and build processes are strictly related to the project and to the environment, it is impossible for this guide to collect and solve all the errors that might be encountered. It is thus recommended that the person approaching this field possesses sufficient problem-solving skills so as to have the ability to deal with the bespoke errors by oneself. A warning that has to be given is that the use of Petalinux and WSL is a little less trivial than using directly a compatible OS. If one chooses to follow this approach, not-well-documented bugs may cause the failure of the processes listed in the following sections.

Chapter 2

Window Subsystems for Linux - WSL

In order to have a working Petalinux environment, a compatible OS among those reported in the file highlighted in Section 1 is needed. However, since not everyone has access to a machine or a server running one of those OS, a valid alternative is to use a virtual machine. There are few possible way to implement such a method, also depending on what kind of operating system one has. In this guide, a solution exploiting Windows Subsystems for Linux is given.

Windows Subsystems for Linux - WSL - is a virtual layer that allows for running a Linux environment on a windows machine without the need to install virtual machine application, such as Virtual Box or VWware, or the need to dual booting. Since it is a Microsoft-native app, it is either already installed in the windows machine to be used (this is the case of Windows 11) or can be installed from Microsoft store.

This implementation, however, is something that will use the computer resources in one of the least efficient way possible, thus the machine will be fully exploited.

0.1 How to...

After the eventual installation, open the Windows Feature and make sure *Virtual Machine Platform* and *Window Subsystems for Linux* (Fig. 0.1) are selected in their respective checkbox and, if not, select them. Click on *OK* and then, if requested, restart the computer. This latter operation may take a while. Now is is possible to install different subsystems. Open the command window - cmd.exe - and type

```
wsl --list --online
```

to get a list of the available OS-LST (Fig. 0.2). After having chosen the needed distribution, it is possible to install it. type

```
wsl --install --OS_NAME
```

to run the installation process. If instead it is typed just

```
wsl --install
```

the latest version of Ubuntu will be installed automatically. After the successful installation it will be required to insert a UNIX username and password. Once they are set, the installation will be completed (Fig. 0.3). To use the terminal of the installed OS open the command prompt of windows, click on the down arrow on the top-left corner, near the "+" sign and choose the shell to be used (Fig. 0.4). A new shell will pop-up and it is now possible to navigate just as if it was a machine running the chosen operating system. To check the correct version of Ubuntu type

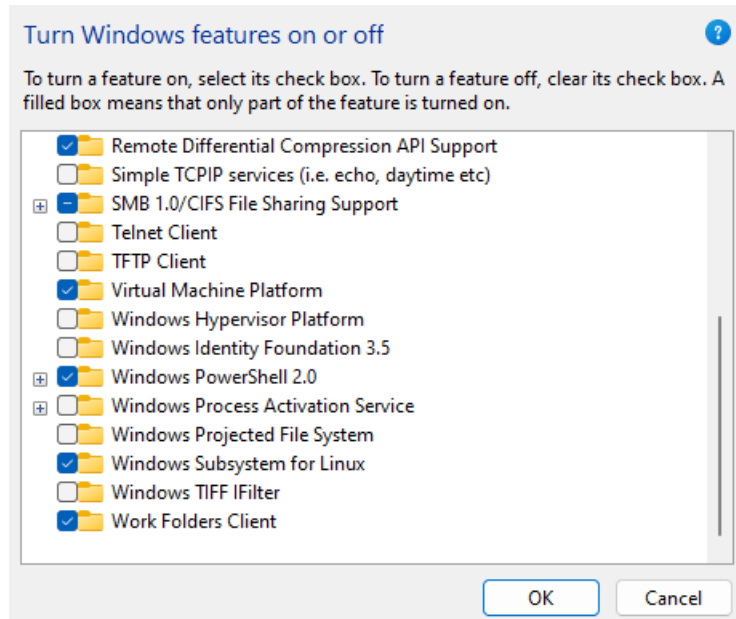


Figure 0.1: Windows Features.

NAME	FRIENDLY NAME
Ubuntu	Ubuntu
Debian	Debian GNU/Linux
kali-linux	Kali Linux Rolling
Ubuntu-18.04	Ubuntu 18.04 LTS
Ubuntu-20.04	Ubuntu 20.04 LTS
Ubuntu-22.04	Ubuntu 22.04 LTS
Ubuntu-24.04	Ubuntu 24.04 LTS
OracleLinux_7_9	Oracle Linux 7.9
OracleLinux_8_7	Oracle Linux 8.7
OracleLinux_9_1	Oracle Linux 9.1
openSUSE-Leap-15.6	openSUSE Leap 15.6
SUSE-Linux-Enterprise-15-SP5	SUSE Linux Enterprise 15 SP5
SUSE-Linux-Enterprise-15-SP6	SUSE Linux Enterprise 15 SP6
openSUSE-Tumbleweed	openSUSE Tumbleweed

Figure 0.2: Available LTS systems.

`lsb_release -a`

and the OS data will be displayed in the prompt window. If the Ubuntu version doesn't appear on the list of Fig. 0.4, restart the terminal.

```
C:\Users\ASUS>wsl --install Ubuntu-20.04
Installing: Ubuntu 20.04 LTS
Ubuntu 20.04 LTS has been installed.
Launching Ubuntu 20.04 LTS...
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: mattia
New password:
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N] y
New password:
Retype new password:
passwd: password updated successfully
The operation completed successfully.
Installation successful!
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

Figure 0.3: Successful installation of the OS.

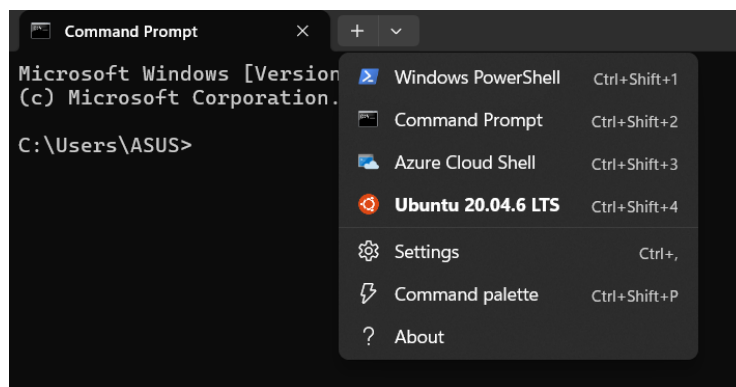


Figure 0.4: Using the shell.

```
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

mattia@Goose:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 20.04.6 LTS
Release:        20.04
Codename:       focal
mattia@Goose:~$
```

Figure 0.5: Ubuntu shell.

Chapter 3

Petalinux - Creating tailor-made OS for FPGAs SoMs

Petalinux is an embedded Software Development Kit - SDK - targeting FPGA-based System on a Chip - SoC - design or FPGA designs.

0.1 Setting up the environment

Before the actual installation of petalinux, some setting of the environment is needed. There are few steps to be followed in order to make the environment work:

- Adding the 32-bit architecture to the system
- Installing the TFTP server (if needed)
- Installing all the required packages for the petalinux Build Host

The first step is taken care of by typing

```
sudo dpkg --add-architecture i386
```

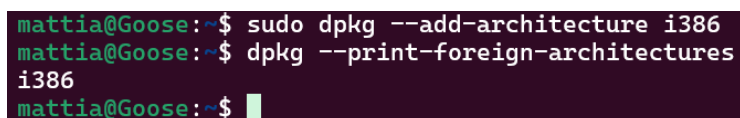
followed by

```
sudo apt-get update
```

This two commands will add the 32-bit architecture and update apt-get, so as to make the changes effective. To see if the architecture has been properly added, type

```
dpkg --print-foreign-architectures
```

and an output like the one of Fig 0.1 will be shown.

A terminal window with a dark background and light green text. It shows three lines of commands and their output. The first line is 'mattia@Goose:~\$ sudo dpkg --add-architecture i386'. The second line is 'mattia@Goose:~\$ dpkg --print-foreign-architectures' followed by the output 'i386' on the next line. The third line is 'mattia@Goose:~\$' followed by a cursor.

```
mattia@Goose:~$ sudo dpkg --add-architecture i386
mattia@Goose:~$ dpkg --print-foreign-architectures
i386
mattia@Goose:~$
```

Figure 0.1: Successful installation of i386 architecture.

The installation of the TFTP server is done by following the subsequent commands.

```
sudo apt-get upgrade
```

```
sudo apt-get update
```

```
mattia@Goose: $ sudo apt-get upgrade
Reading package lists... Done
Building dependency tree
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
mattia@Goose: $ sudo apt-get update
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Hit:2 http://archive.ubuntu.com/ubuntu focal InRelease
Get:3 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:4 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [3304 kB]
Get:5 http://security.ubuntu.com/ubuntu focal-security/main i386 Packages [835 kB]
Get:6 http://archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Get:7 http://archive.ubuntu.com/ubuntu focal-updates/main i386 Packages [1056 kB]
Get:8 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages [3681 kB]
Get:9 http://archive.ubuntu.com/ubuntu focal-updates/restricted amd64 Packages [3397 kB]
Get:10 http://archive.ubuntu.com/ubuntu focal-updates/restricted i386 Packages [40.4 kB]
Get:11 http://archive.ubuntu.com/ubuntu focal-updates/restricted Translation-en [474 kB]
Get:12 http://archive.ubuntu.com/ubuntu focal-updates/multiverse amd64 Packages [27.9 kB]
Get:13 http://archive.ubuntu.com/ubuntu focal-updates/multiverse Translation-en [7968 B]
Fetched 13.2 MB in 3s (5243 kB/s)
Reading package lists... Done
mattia@Goose: $
```

Figure 0.2: Apt upgrading and updating.

and a window similar to that of Figure 0.2 will appear. Now it is possible to actually install the TFTP server. Type

```
sudo apt-get install tftpd-hpa
```

and let the computer run the respective task. Once it is done, check for the correct installation typing

```
sudo systemctl status tftpd-hpa.service
```

and the output of Figure 0.3 should appear. To start the server automatically during the boot, type

```
sudo systemctl enable tftpd-hpa
```

```
mattia@Goose: $ sudo systemctl status tftpd-hpa.service
● tftpd-hpa.service - LSB: HPA's tftp server
   Loaded: loaded (/etc/init.d/tftpd-hpa; generated)
   Active: active (running) since Mon 2024-12-09 18:30:41 CET; 2min 26s ago
     Docs: man:systemd-sysv-generator(8)
    Tasks: 1 (limit: 4695)
   CGroup: /system.slice/tftpd-hpa.service
           └─2321 /usr/sbin/in.tftpd --listen --user tftp --address :69 --secure /var/lib/tftpboot

Dec 09 18:30:41 Goose systemd[1]: Starting LSB: HPA's tftp server...
Dec 09 18:30:41 Goose tftpd-hpa[2313]: * Starting HPA's tftp in.tftpd
Dec 09 18:30:41 Goose tftpd-hpa[2313]:   ...done.
Dec 09 18:30:41 Goose systemd[1]: Started LSB: HPA's tftp server.
mattia@Goose: $ sudo systemctl enable tftpd-hpa
tftpd-hpa.service is not a native service, redirecting to systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable tftpd-hpa
mattia@Goose: $
```

Figure 0.3: Correctly running TFTP server.

Now the server should be all set. After the installation, it is necessary to configure the server. Type

```
sudo nano /etc/default/tftpd-hpa
```

and in the window that will open write down the following text

```
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/var/lib/tftpboot"
TFTP_ADDRESS="0.0.0.0:69"
TFTP_OPTIONS="--secure"
```

as shown in Figure 0.4. Then, finish the configuration with the following commands (one per time)



```
GNU nano 4.8 /etc/default/tftpd-hpa
TFTP_USERNAME='tftp'
TFTP_DIRECTORY='/var/lib/tftpboot'
TFTP_ADDRESS='0.0.0.0:69'
TFTP_OPTIONS='--secure'
```

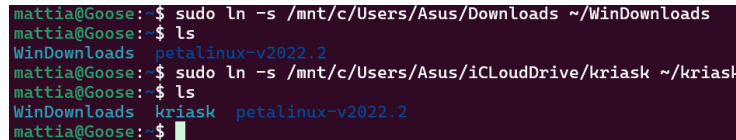
Figure 0.4: Setting TFTP server.

```
sudo mkdir -p /var/lib/tftpboot
sudo chown -R nobody:nogroup /var/lib/tftpboot
sudo chmod -R 777 /var/lib/tftpboot
sudo systemctl restart tftpd-hpa
```

A trick that will come in hand is to link the Windows downloads folder and the Vivado workspace with a custom-made folder within the WSL. To do so type the following command in the WSL shell (on the same line)

```
sudo ln -s /mnt/c/Users/<folder_path>/<folder_name> ~/<new_folder_name>
```

The results will be the one shown in Figure 0.5



```
mattia@Goose:~$ sudo ln -s /mnt/c/Users/Asus/Downloads ~/WinDownloads
mattia@Goose:~$ ls
WinDownloads  petalinux-v2022.2
mattia@Goose:~$ sudo ln -s /mnt/c/Users/Asus/iCloudDrive/kriask ~/kriask
mattia@Goose:~$ ls
WinDownloads  kriask  petalinux-v2022.2
mattia@Goose:~$
```

Figure 0.5: Successful link created.

Finally, all the needed packages can be installed. From this page, download the file at the bottom. Create the folder in which petalinux will be installed with the command

```
mkdir <folder_name>
```

In this tutorial it is named *petalinux-v2022.2* (see Figure 0.5). Copy the file from the download folder to the bespoke petalinux installation folder by typing

```
cp <start_path> <final_path>
```

and then type

```
chmod +x <installer_file_name>
sudo ./<file_name>
```

Once the installation is complete, all the packages should have been installed. it might be necessary to reinstall the tftpd-hpa server after this operation.

Now it is possible to begin the actual installation of petalinux. From the petalinux AMD download page, download the needed version and move it to the installation folder. From this page also download the .BPS file relative to the board to be used. This latter file has to be moved into the folder in which the petalinux project are saved. Now type

```
chmod +x <installer_file_name>
./<file_name>
```

if an error message appears (Fig. 0.6), then type

```
sudo chmod 666 petalinux_installation_log
./<installer_file_name>
```

```
./petalinux-v2022.2-10141622-installer.run: line 91: /home/mattia/petalinux22/petalinux_installation_log: Permission denied
tee: /home/mattia/petalinux22/petalinux_installation_log: Permission denied
INFO: Checking installation environment requirements...

tee: /home/mattia/petalinux22/petalinux_installation_log: Permission denied
Please refer to the Petalinux Tools Installation Guide.

Check the troubleshooting guide at the end of that manual, and if you are
unable to resolve the issue please contact customer support with file:
/home/mattia/petalinux22/petalinux_installation_log

./petalinux-v2022.2-10141622-installer.run: line 175: /tmp/tmp.2TrGMUeksq/initsetup.tar.xz: No such file or directory
truncate: cannot open '/tmp/tmp.2TrGMUeksq/initsetup.tar.xz' for writing: No such file or directory
tar: /tmp/tmp.2TrGMUeksq/initsetup.tar.xz: Cannot open: No such file or directory
tar: Error is not recoverable: exiting now
chmod: cannot access '/tmp/tmp.2TrGMUeksq/petalinux-env-check': No such file or directory
tee: /home/mattia/petalinux22/petalinux_installation_log: Permission denied
./petalinux-v2022.2-10141622-installer.run: line 235: /tmp/tmp.2TrGMUeksq/petalinux-env-check: No such file or directory
tee: /home/mattia/petalinux22/petalinux_installation_log: Permission denied
WARNING: Please install required packages.
```

Figure 0.6: Output error.

Now the installation should begin. By following the instruction, the installation process should proceed without any inconvenience, even though it might take a while. Finally, to set the petalinux environment into the directory in which it is installed, type

```
source settings.sh
```

from which the shell output highlighted in Fig. 0.7 is shown. Now the system should be

```
mattia@Goose:~/petalinux-v2022.2$ source settings.sh
Petalinux environment set to '/home/mattia/petalinux-v2022.2'
WARNING: /bin/sh is not bash!
bash is Petalinux recommended shell. Please set your default shell to bash.
WARNING: This is not a supported OS
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
mattia@Goose:~/petalinux-v2022.2$
```

Figure 0.7: Setting petalinux environment.

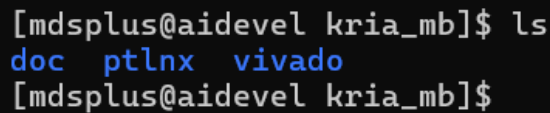
all set. The [WARNING] message relative to the OS system is due to the used version of Ubuntu, that is Ubuntu 20.04.6, which is not among the supported OS listed by AMD.

Due to the high load that the following process will constitute for a normal device, the guide from now on will refer to a server running CentOS Stream 9. All the procedure, however, should be the same, except minor errors or problems.

Chapter 4

Petalinux project: from start to bottom

Now that all the tools and utilities are installed and the environment is ready, it is possible to create the first petalinux project. The environment created on the server is the one shown in Figure 0.1 in which the folder *doc* is used to store all the documents needed for

A terminal window with a black background and white text. The prompt is [mdsplus@aidevel kria_mb]\$. The command ls is entered, and the output is doc ptlnx vivado. The prompt is repeated on the next line.

```
[mdsplus@aidevel kria_mb]$ ls
doc ptlnx vivado
[mdsplus@aidevel kria_mb]$
```

Figure 0.1: Environment set-up.

the project creation, the folder *vivado* will contain all the Vivado project and the folder *ptlnx* is the installation folder of petalinux-v2022.2.

A prerequisite is having already generated the bitstream and exported the hardware description from the Vivado project.

0.1 Building the project

From the AMD download page, download the .BSP file of the board it is about to be used. Inside petalinux installation folder, create another folder named prj. (here all the projects will be created, build and saved) and copy the .BSP file in this directory. Move now into this folder. As the first task, upgrade petalinux eSDK typing (everything on the same line)

```
petalinux-upgrade -u http://petalinux.xilinx.com/sswreleases/rel-v2022
/sdkupdate/2022.2_update1/ -p "aarch64" --wget-args "--wait 1 -nH --cut-dirs=4"
```

To create the project, type

```
petalinux-create --type project -s <BPS_FILE_PATH> --name <PROJECT_NAME>
```

and move into the created folder. The next step is to configure the project with the following command line (the *HARDWARE_FILE_PATH* is the file with the extension .xsa in the Vivado project folder)

```
petalinux-config --get-hw-description <HARDWARE_FILE_PATH>
```

after which a window like the one of Figure 0.2. Following the instructions, navigate into the folders below reported and change the respective configurations:

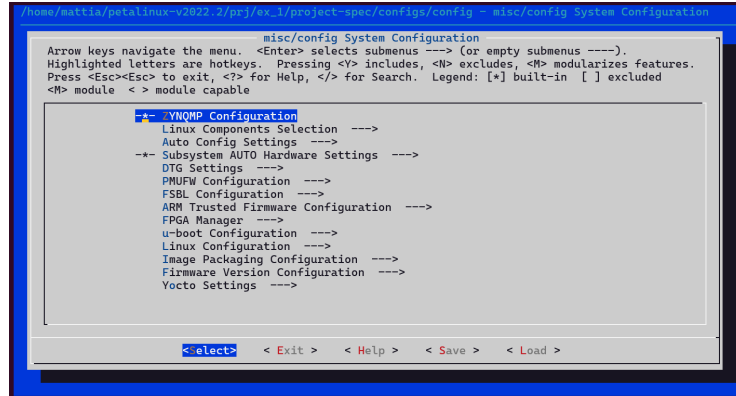


Figure 0.2: Petalinux configuration window.

- *FPGA Manager* -- > *Fpga Manager[*]*
- *Image Packaging Configuration* -- > *Root Filesystem Type* -> *INITRD[*]*
- *Image Packaging Configuration* -- > *INITRAMFS/INITRD Image name* -- > *petalinux-initramfs-image*
- *Image Packaging Configuration* -- > *Copy final images to tftpboot[]*

Exit and save the project. It is possible now to build the project. Type

```
petalinux-build
```

and let the machine to run. After this operation, it is possible to add, if needed, various kernel packages typing

```
petalinux-config -c kernel
```

The next step is to configure the root file system (see Figure 0.3) with the command

```
petalinux-config -c rootfs
```

from which all the needed packages are installed. An example fo the packages installed ca be (acceleration, XRT and gcc):

- Filesystem Packages -- > console -- > utils -- > git -- > [*] git
- Filesystem Packages -- > base -- > dnf -- > [*] dnf
- Filesystem Packages -- > x11 -- > base -- > libdrm -- > [*] libdrm
- Filesystem Packages -- > x11 -- > base -- > libdrm -- > [*] libdrm-tests
- Filesystem Packages -- > x11 -- > base -- > libdrm -- > [*] libdrm-kms
- Filesystem Packages -- > libs -- > xrt -- > [*] xrt
- Filesystem Packages -- > libs -- > xrt -- > [*] xrt-dev
- Filesystem Packages -- > libs -- > zocl -- > [*] zocl
- Filesystem Packages -- > libs -- > opencl-headers -- > [*] opencl-headers

- Filesystem Packages -- > libs -- > opencl-clhpp -- > [*] opencl-clhpp-dev
- Filesystem Packages -- > misc -- > packagegroup-core-buildessential -- > packagegroup-core-buildessential[*]
- Filesystem Packages -- > misc -- > packagegroup-core-buildessential -- > packagegroup-core-buildessential[*]
- Filesystem Packages -- > misc -- > gcc-runtime -- > libstdc++[*]
- Petalinux Package Groups -- > packagegroup-petalinux -- > [*] packagegroup-petalinux
- Petalinux Package Groups -- > packagegroup-petalinux-gstreamer -- > [*] packagegroup-petalinux-gstreamer
- Petalinux Package Groups -- > packagegroup-petalinux-opencv -- > [*] packagegroup-petalinux-gstreamer
- Petalinux Package Groups -- > packagegroup-petalinux-v4lutils -- > [*] packagegroup-petalinux-gstreamer
- Petalinux Package Groups -- > packagegroup-petalinux-x11 -- > [*] packagegroup-petalinux-gstreamer

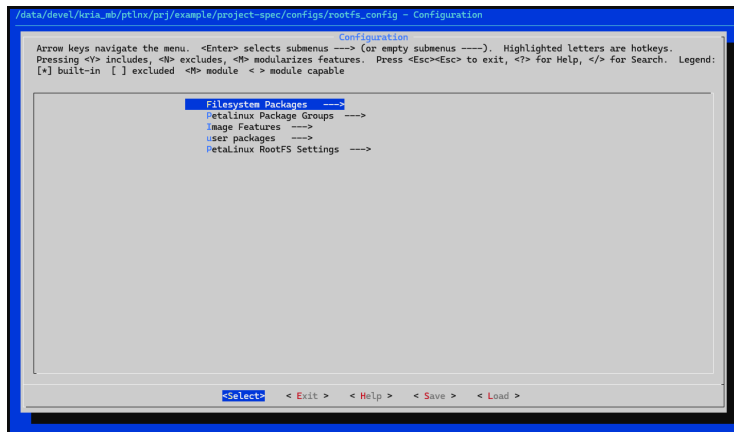


Figure 0.3: Petalinux root configuration window.

After exiting the window, rebuild the project. The last step in order to finish the project creation is to build the SDK with the command

```
petalinux-build --sdk
```

0.2 Preparing the SD-card

There are two possible ways to prepare the SD-card for the KRIA KR260: create the .wic file to be flashed directly into the SD or to create the partitions on the SD-card and upload the needed files in the respective partitions. The first step is, anyway, to generate the images

```
petalinux-package --boot --u-boot --force
```

.wic file creation

To generate the .wic file. type into the command line (on the same line)

```
petalinux-package --wic --images-dir images/linux/ --bootfiles  
"ramdisk.cpio.gz.u-boot,boot.scr,Image,system.dtb,system-zynqmp-sck-kr-g-revB.dtb"  
--disk-name "sda"
```

and let the machine to create the file. After that it is possible to flash it to the SD-card with Balena Etcher (or another program just alike).

SD-card partitioning

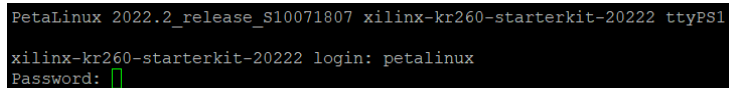
Another possible, but a bit more complex, way to boot the KR260 from SD is to partition the SD card and move into the apposite partitions the needed images. To fully understand this method, please refer to this page.

Warning: approaching this method from WSL is not possible. There are some problems between the interface of the SD cards / USB connectors and the virtual machine.

0.3 Booting the board

To boot the board it is simply needed to insert the SD-card into the apposite slot, connect the power and the USB cable to a computer. To read the data stream, download a terminal application like PuTTY. After the boot, it is necessary to log in the system. It will request a username and a password:

```
username: petalinux  
passwd: your_password
```



```
PetaLinux 2022.2_release_S10071807 xilinx-kr260-starterkit-20222 ttyPS1  
xilinx-kr260-starterkit-20222 login: petalinux  
Password: █
```

Figure 0.4: Booting board.